

A* Search Algorithm

Aim

To implement the A* Search Algorithm to find the optimal path from the start node A to the goal node G using heuristic values.

Problem Statement

Graph

$$A \rightarrow B = 2$$

$$A \rightarrow C = 4$$

$$B \rightarrow C = 3$$

$$B \rightarrow D = 7$$

$$B \rightarrow E = 2$$

$$C \rightarrow E = 3$$

$$D \rightarrow E = 2$$

$$E \rightarrow G = 2$$

Heuristic Values

Node $h(n)$

A 7

B 6

C 4

D 3

E 2

G 0

Algorithm

1. Start from the source node A.
2. Initialize the Open List with the start node.
3. Calculate:
 $g(n)$ = Cost from start node.
 $h(n)$ = Heuristic value.
 $f(n) = g(n) + h(n)$.
4. Select the node with the minimum $f(n)$.
5. Move it to the Closed List.
6. Expand all neighboring nodes.
7. Update $g(n)$, $h(n)$, and $f(n)$ if a better path is found.
8. Repeat until the goal node G is reached.
9. Print the optimal path and total cost.

Dry Run

Iteration 1

Current Node: A

Node	$g(n)$	$h(n)$	$f(n)$
A	0	7	7

B	2	6	8
C	4	4	8

Open List: B(8), C(8)

Closed List: A

Iteration 2

Current Node: B

Node	$g(n)$	$h(n)$	$f(n)$
E	4	2	6
C	4	4	8
D	9	3	12

Open List: E(6), C(8), D(12)

Closed List: A, B

Iteration 3

Current Node: E

Node	$g(n)$	$h(n)$	$f(n)$
G	6	0	6
C	4	4	8
D	6	3	9

Open List: G(6), C(8), D(9)

Closed List: A, B, E

Iteration 4

Current Node: G

Goal Node Reached.

Closed List: A, B, E, G

Python Code

```
import heapq

graph = {
    'A': [('B', 2), ('C', 4)],
    'B': [('C', 3), ('D', 7), ('E', 2)],
    'C': [('E', 3)],
    'D': [('E', 2)],
    'E': [('G', 2)],
    'G': []
}

heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 3,
    'E': 2,
    'G': 0
}

def astar(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristic[start], start))

    g = {node: float('inf') for node in graph}
    g[start] = 0

    parent = {start: None}
    closed_list = []

    while open_list:
```

```

f, current = heapq.heappop(open_list)

if current in closed_list:
    continue

print("\nCurrent Node:", current)
closed_list.append(current)

if current == goal:
    path = []
    while current:
        path.append(current)
        current = parent[current]
    path.reverse()

    print("\nOptimal Path:", " -> ".join(path))
    print("Total Path Cost:", g[goal])
    return

for neighbor, cost in graph[current]:
    new_g = g[current] + cost

    if new_g < g[neighbor]:
        g[neighbor] = new_g
        parent[neighbor] = current
        f_cost = new_g + heuristic[neighbor]
        heapq.heappush(open_list, (f_cost, neighbor))

print("\nOpen List:")
for item in sorted(open_list):
    node = item[1]
    print(f"{node}: g={g[node]}, h={heuristic[node]}, f={g[node] + heuristic[node]}")

print("Closed List:", closed_list)

astar(graph, heuristic, 'A', 'G')

---
```

Expected Output

Current Node: A

Open List:

B: g=2, h=6, f=8

C: g=4, h=4, f=8

Closed List: ['A']

Current Node: B

Open List:

E: g=4, h=2, f=6

C: g=4, h=4, f=8

D: g=9, h=3, f=12

Closed List: ['A', 'B']

Current Node: E

Open List:

G: g=6, h=0, f=6

C: g=4, h=4, f=8

D: g=6, h=3, f=9

Closed List: ['A', 'B', 'E']

Current Node: G

Optimal Path: A -> B -> E -> G

Total Path Cost: 6

Result

Optimal Path: A → B → E → G

Total Path Cost: 6

```

1 import heapq
2
3 graph = {
4     'A': [['B', 3], ['C', 4]],
5     'B': [['C', 3], ['D', 7], ['E', 3]],
6     'C': [['E', 3]],
7     'D': [['E', 3]],
8     'E': [['G', 3]],
9     'G': []
10 }
11
12 heuristic = {
13     'A': 5,
14     'B': 4,
15     'C': 4,
16     'D': 3,
17     'E': 2,
18     'G': 0
19 }
20
21 def astar(graph, heuristic, start, goal):
22     open_list = []
23     heapq.heappush(open_list, (heuristic[start],
24                               start))
25
26     g = {node: float('inf') for node in graph}
27     g[start] = 0
28     parent = {start: None}
29     closed_list = []
30
31     while open_list:
32         f, current = heapq.heappop(open_list)
33
34         if current in closed_list:
35             continue
36
37         print(f"Current Node: {current}")
38         closed_list.append(current)
39
40         if current == goal:
41             path = []
42             while current:
43                 path.append(current)
44                 current = parent[current]
45             path.reverse()
46             print(f"Optimal Path: { ' -> '.join(path)}")
47             print(f"Total Path Cost: {g[goal]}")
48             return
49
50         for neighbor, cost in graph[current]:
51             new_g = g[current] + cost
52
53             if new_g < g[neighbor]:
54                 g[neighbor] = new_g
55                 parent[neighbor] = current
56                 f_cost = new_g + heuristic[neighbor]
57                 heapq.heappush(open_list, (f_cost, neighbor))
58
59         print(f"Open List:")
60         for item in sorted(open_list):
61             node = item[1]
62             print(f"({node}) g={g[node]}, h={heuristic[node]}, f={g[node] + heuristic[node]}")
63
64         print(f"Closed List: {closed_list}")
65
66     astar(graph, heuristic, 'A', 'G')

```

Output:

```

Current Node: A
Open List:
B: g=2, h=4, f=6
C: g=4, h=4, f=8
Closed List: ['A']
Current Node: B
Open List:
E: g=4, h=2, f=6
C: g=4, h=4, f=8
D: g=9, h=3, f=12
Closed List: ['A', 'B']
Current Node: E
Open List:
G: g=4, h=0, f=4
D: g=9, h=3, f=12
Closed List: ['A', 'B', 'E']
Current Node: G
Optimal Path: A -> B -> E -> G
Total Path Cost: 6
--- Code Execution Successful ---

```

Formula Used:

$g(n)$ = Cost from start node to current node

$h(n)$ = Heuristic value

$f(n) = g(n) + h(n)$

Question 2: Minimax with Alpha-Beta Pruning

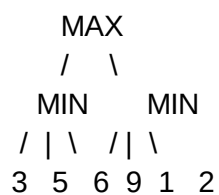
Aim

To implement the Minimax Algorithm with Alpha-Beta Pruning and determine the best move for the MAX player.

Algorithm

1. Start from the root (MAX node).
2. MAX chooses the maximum value, while MIN chooses the minimum value.
3. Initialize:
 Alpha (α) = $-\infty$
 Beta (β) = $+\infty$
4. Traverse the tree from left to right.
5. Update α and β after evaluating each node.
6. If $\alpha \geq \beta$, prune the remaining branches.
7. Continue until the entire tree is evaluated.
8. Return the optimal move and minimax value.

Given Tree



Dry Run

Initial Values

Root (MAX): $\alpha = -\infty$, $\beta = +\infty$

Step 1: Evaluate Left MIN

Initial:

$$\alpha = -\infty$$

$$\beta = +\infty$$

Evaluate leaves:

$$3 \rightarrow \beta = \min(+\infty, 3) = 3$$

$$5 \rightarrow \beta = \min(3, 5) = 3$$

$$6 \rightarrow \beta = \min(3, 6) = 3$$

Selected Value = 3

Return 3 to MAX.

Update MAX:

$$\alpha = \max(-\infty, 3) = 3$$

Step 2: Evaluate Right MIN

Current:

$$\alpha = 3$$

$$\beta = +\infty$$

Evaluate first leaf:

$$9 \rightarrow \beta = \min(+\infty, 9) = 9$$

Since α (3) < β (9), continue.

Evaluate second leaf:

$$1 \rightarrow \beta = \min(9, 1) = 1$$

Now,

$$\alpha = 3$$

$$\beta = 1$$

Since $\alpha \geq \beta$ ($3 \geq 1$),

Prune the remaining node (2).

Selected Value = 1

Return 1 to MAX.

Step 3: MAX Node

Compare returned values:

Left MIN = 3

Right MIN = 1

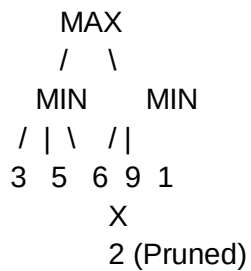
MAX chooses the larger value.

Selected Value = 3

Alpha-Beta Table

Node	α	β	Selected Value	Pruned
Left MIN	$-\infty$	3	3.	None
MAX	3	$+\infty$	3	None
Right MIN	3	1	1	Node 2
MAX (Final)	3	$+\infty$	3	None

Tree After Pruning



Final Answer

Best Move for MAX

Choose the left MIN subtree.

Final Minimax Value

3

Pruned Node

Node 2

Result

Best Move: Left MIN subtree

Final Minimax Value: 3

Pruned Node: 2

Code

```
import math

# Given game tree leaf values
#      MAX
#    /   \
#  MIN   MIN
# /|\  /|\
# 3 5 6 9 1 2

tree = [
    [3, 5, 6], # Left MIN node
    [9, 1, 2]  # Right MIN node
]

def minimax(values, depth, is_max, alpha, beta):

    if depth == 1:
        return values

    if is_max:
        best = -math.inf

        for value in values:
            best = max(best, value)
            alpha = max(alpha, best)

            if beta <= alpha:
                break

        return best

    else:
        best = math.inf

        for value in values:
            best = min(best, value)
            beta = min(beta, best)

            if beta <= alpha:
                break
```

```

        return best

# Initial values
alpha = -math.inf
beta = math.inf

# Evaluate left MIN node
left_value = min(tree[0])

alpha = max(alpha, left_value)

print("Left MIN value:", left_value)
print("Alpha after left subtree:", alpha)

# Evaluate right MIN node with alpha-beta pruning
right_value = math.inf

for i, value in enumerate(tree[1]):
    right_value = min(right_value, value)
    beta = min(beta, right_value)

    if beta <= alpha:
        print("Pruned node:", tree[1][i+1:])
        break

print("Right MIN value:", right_value)

# MAX chooses maximum
minimax_value = max(left_value, right_value)

print("\nFinal Minimax Value:", minimax_value)
print("Best Move for MAX: Left MIN subtree")

```

Output

```

Left MIN value: 3
Alpha after left subtree: 3
Pruned node: [2]
Right MIN value: 1

```

```

Final Minimax Value: 3
Best Move for MAX: Left MIN subtree

```

Programiz
Python Online Compiler

Programiz PRO

main.py

Run

Clear

Output

```
1 import math
2
3 # Given game tree leaf values
4 #      max
5 #      / \
6 #     MIN MIN
7 #    /  \  \
8 #   3  5  4  5  1  2
9
10 tree = [
11     [3, 5, 4], # Left MIN node
12     [5, 1, 2]  # Right MIN node
13 ]
14
15 def minimax(values, depth, is_max, alpha, beta):
16
17     if depth == 1:
18         return values
19
20     if is_max:
21         best = -math.inf
22
23         for value in values:
24             best = max(best, value)
25             alpha = max(alpha, best)
26
27             if beta == alpha:
28                 break
29
30         return best
31
32     else:
33         best = math.inf
34
35         for value in values:
36             best = min(best, value)
37             beta = min(beta, best)
38
39             if beta == alpha:
40                 break
41
42         return best
43
44 # Initial values
45 alpha = -math.inf
46 beta = math.inf
47
48 # Evaluate left MIN node
49 left_value = min(tree[0])
50
51 alpha = max(alpha, left_value)
52
53 print("Left MIN value:", left_value)
54 print("Alpha after left subtree:", alpha)
55
56 # Evaluate right MIN node with alpha-beta pruning
57 right_value = math.inf
58
59 for i, value in enumerate(tree[1]):
60     right_value = min(right_value, value)
61     beta = min(beta, right_value)
```

Left MIN value: 3
Alpha after left subtree: 3
Pruned node: [2]
Right MIN value: 1
Final Minimax Value: 3
Best Move for MAX: Left MIN subtree
=== Code Execution Successful ===